November 03rd, 2023

Course: Computer Graphics

Instructor: Assoc. Prof. Nguyen Van Sinh

Email: nvsinh@hcmiu.edu.vn

Lab 3: Advance graphic in 2D, Transformation, Animation

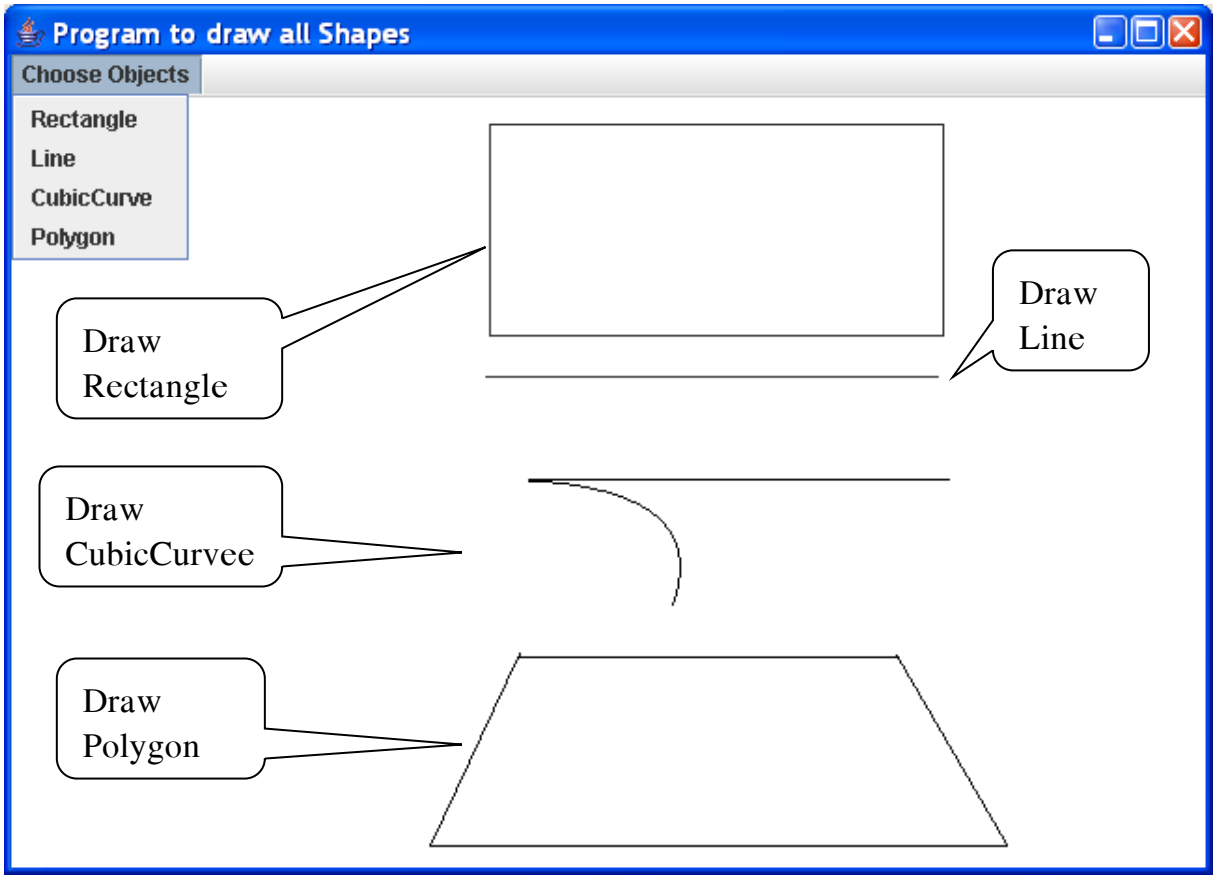
Contents:

- **Advance graphic in 2D**
- **Transformation**
- **Animation**
- **Exercises**

Duration: 3 hours

Part 1: Advance graphic in 2D

In this part, we will create a java application "Program to draw all Shapes". There is a menu "Choose Objects", when user click here, she/he can choose one of the objects in the dropdown menu to draw.



This is the source code of program:

```
public class DrawShapes extends JApplet implements ActionListener {
  public static void main(String s[]) {
    JFrame frame = new JFrame();
    frame.setTitle("Program to draw all Shapes");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    JApplet applet = new DrawShapes();
    applet.init();
    frame.getContentPane().add(applet);
    frame.pack();
    frame.setVisible(true);
  }

  JavaDraw2DPanel panel = null;

  public void init() {                    //this method to create menu
    JMenuBar mb = new JMenuBar();
    setJMenuBar(mb);
    JMenu menu = new JMenu("Choose Objects");
    mb.add(menu);
    JMenuItem mi = new JMenuItem("Rectangle");
    mi.addActionListener(this);
    menu.add(mi);
    mi = new JMenuItem("Line");
    mi.addActionListener(this);
    menu.add(mi);
    mi = new JMenuItem("CubicCurve");
    mi.addActionListener(this);
    menu.add(mi);
    mi = new JMenuItem("Polygon");
    mi.addActionListener(this);
    menu.add(mi);
    panel = new JavaDraw2DPanel();
    getContentPane().add(panel);
  }

  public void actionPerformed(ActionEvent ev) {
    String command = ev.getActionCommand();
    if ("Rectangle".equals(command)) {
      panel.shapeType = panel.RECTANGLE;
    } else if ("Line".equals(command)) {
      panel.shapeType = panel.LINE2D;
    } else if ("CubicCurve".equals(command)) {
      panel.shapeType = panel.CUBICCURVE2D;
    } else if ("Polygon".equals(command)) {
      panel.shapeType = panel.POLYGON;
    }
  }
}

class    JavaDraw2DPanel    extends    JPanel    implements    MouseListener,
MouseMotionListener {
  private Vector shapes = new Vector();
  static final int RECTANGLE = 0;
  static final int LINE2D = 1;
  static final int CUBICCURVE2D = 2;
  static final int POLYGON = 3;
  static final int GENERAL = 4;
```

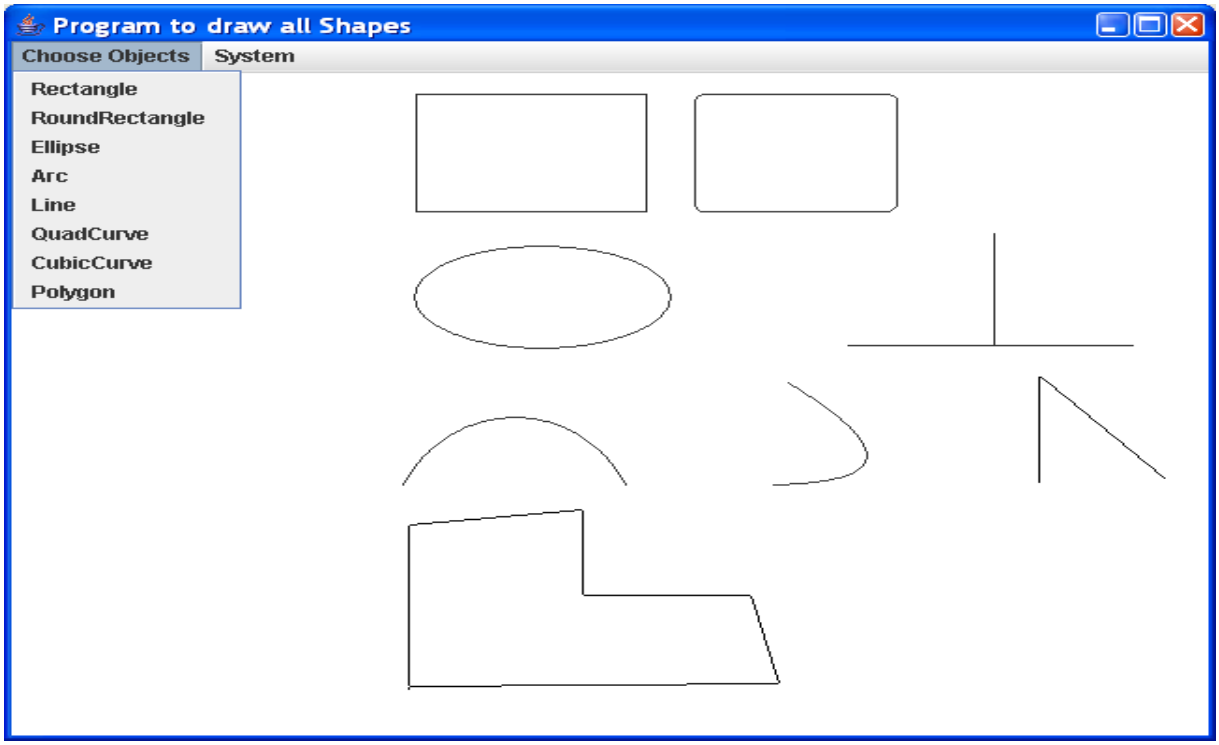
```
  static final int AREA = 5;

  int shapeType = RECTANGLE;
  // vector of input points
  Vector points = new Vector();
  int pointIndex = 0;
  Shape partialShape = null;
  Point p = null;

  public JavaDraw2DPanel() {
    super();
    setBackground(Color.white);
    setPreferredSize(new Dimension(640, 480));
    addMouseListener(this);
    addMouseMotionListener(this);
  }

  public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D)g;
    for (int i = 0; i < shapes.size(); i++) {
      Shape s = (Shape)shapes.get(i);
      g2.draw(s);
    }
  }

  public void mouseClicked(MouseEvent ev) {
  }

  public void mouseEntered(MouseEvent ev) {
  }

  public void mouseExited(MouseEvent ev) {
  }

  public void mousePressed(MouseEvent ev) {
    points.add(ev.getPoint());
    pointIndex++;
    p = null;
  }

  public void mouseReleased(MouseEvent ev) {
    Graphics g = getGraphics();
    Point p1 = (Point)(points.get(pointIndex-1));
    p = ev.getPoint();
    Shape s = null;
    switch (shapeType) {
      case RECTANGLE:
        s = new Rectangle(p1.x, p1.y, p.x-p1.x, p.y-p1.y);
        break;
      case LINE2D:
        s = new Line2D.Float(p1.x, p1.y, p.x, p.y);
        break;
      case CUBICCURVE2D:
        if (pointIndex > 2) {
          Point p2 = (Point)points.get(pointIndex-2);
          Point p3 = (Point)points.get(pointIndex-3);
          s = new CubicCurve2D.Float(p3.x, p3.y, p2.x, p2.y, p1.x, p1.y, p.x,
p.y);
        }
        break;
```

```
      case POLYGON:
        if (ev.isShiftDown()) {
          s = new Polygon();
          for     (int     i     =     0;     i     <     pointIndex;     i++)
((Polygon)s).addPoint(((Point)points.get(i)).x, ((Point)points.get(i)).y);
          ((Polygon)s).addPoint(p.x, p.y);
        }

    }
    if (s != null) {
      shapes.add(s);
      points.clear();
      pointIndex = 0;
      p = null;
      repaint();
    }
  }

  public void mouseMoved(MouseEvent ev) {
  }

  public void mouseDragged(MouseEvent ev) {
    Graphics2D g = (Graphics2D)getGraphics();
    g.setXORMode(Color.white);
    Point p1 = (Point)points.get(pointIndex-1);
    switch (shapeType) {
      case RECTANGLE:
        if (p != null) g.drawRect(p1.x, p1.y, p.x-p1.x, p.y-p1.y);
        p = ev.getPoint();
        g.drawRect(p1.x, p1.y, p.x-p1.x, p.y-p1.y);
        break;
      case LINE2D:
      case POLYGON:
        if (p != null) g.drawLine(p1.x, p1.y, p.x, p.y);
        p = ev.getPoint();
        g.drawLine(p1.x, p1.y, p.x, p.y);
        break;
      case CUBICCURVE2D:
        if (pointIndex == 1) {
          if (p != null) g.drawLine(p1.x, p1.y, p.x, p.y);
          p = ev.getPoint();
          g.drawLine(p1.x, p1.y, p.x, p.y);
        } else if (pointIndex == 2) {
          Point p2 = (Point)points.get(pointIndex-2);
          if (p != null) g.draw(partialShape);
          p = ev.getPoint();
          partialShape = new QuadCurve2D.Float(p2.x, p2.y, p1.x, p1.y, p.x,
p.y);
          g.draw(partialShape);
        } else {
          Point p2 = (Point)points.get(pointIndex-2);
          Point p3 = (Point)points.get(pointIndex-3);
          if (p != null) g.draw(partialShape);
          p = ev.getPoint();
          partialShape = new CubicCurve2D.Float(p3.x, p3.y, p2.x, p2.y, p1.x,
p1.y, p.x, p.y);
          g.draw(partialShape);
        }
        break;
    }
  }
```

```
}
```

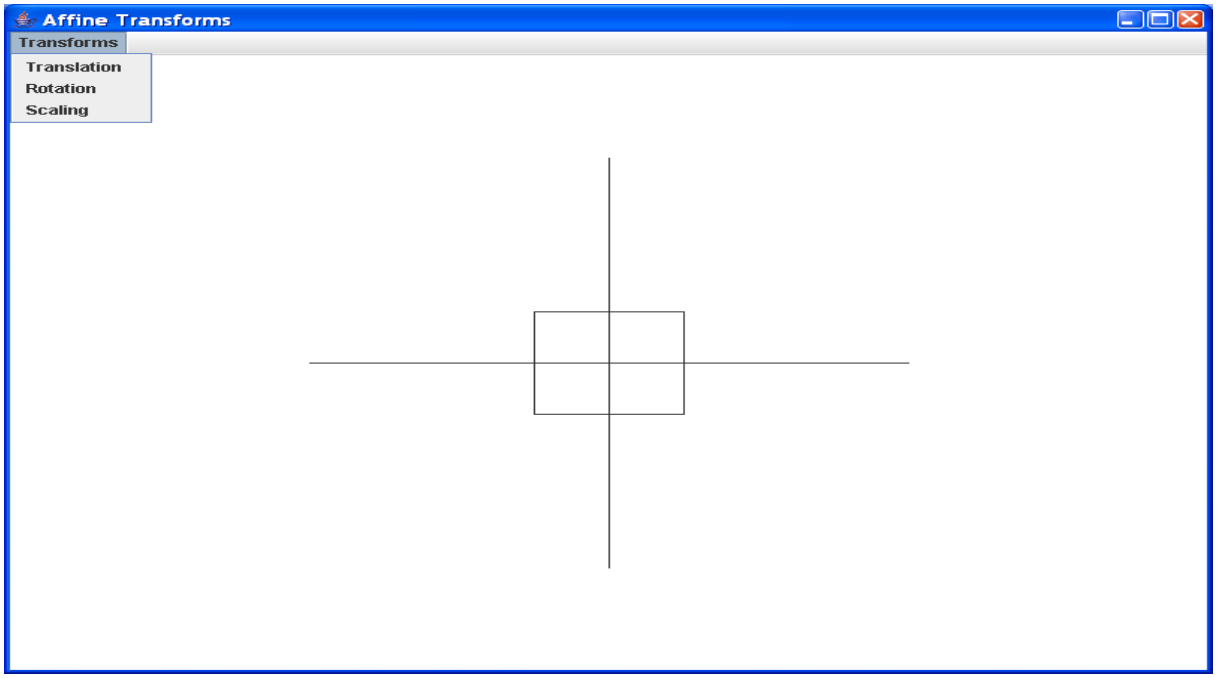
Exercise 1: Copy the above source code and modify:

- Add one more menu "System", when click here there is an Item "Close" to exit program.
- Add more 4 objects (RoundRectangle, Ellipse, Arc, QuadCurve) onto the menu "Choose Objects" and therefore user can draw them. The result is as below figure:



Part 2: Transformation

In this part, we will create a java application "Affine Transform". There is a menu "Transforms", when user click here, they can choose the objects in the dropdown menu to translate, rotate and scale.



This is the source code of program:

```
public class Transformation extends JApplet implements ActionListener {

    public static void main(String args[]) {
        JFrame frame = new JFrame();
        frame.setTitle("Affine Transforms");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JApplet applet = new Transformation();
        applet.init();
        frame.getContentPane().add(applet);
        frame.pack();
        frame.setVisible(true);
    }
    TransformPanel panel = null;

    public void init() {
        JMenuBar mb = new JMenuBar();
        setJMenuBar(mb);
        JMenu menu = new JMenu("Transforms");
        mb.add(menu);
        JMenuItem mi = new JMenuItem("Translation");
        mi.addActionListener(this);
        menu.add(mi);
        mi = new JMenuItem("Rotation");
        mi.addActionListener(this);
        menu.add(mi);
        mi = new JMenuItem("Scaling");
        mi.addActionListener(this);
        menu.add(mi);

        panel = new TransformPanel();
        getContentPane().add(panel);
    }

    public void actionPerformed(ActionEvent ev) {
        String command = ev.getActionCommand();
        if ("Translation".equals(command)) {
```

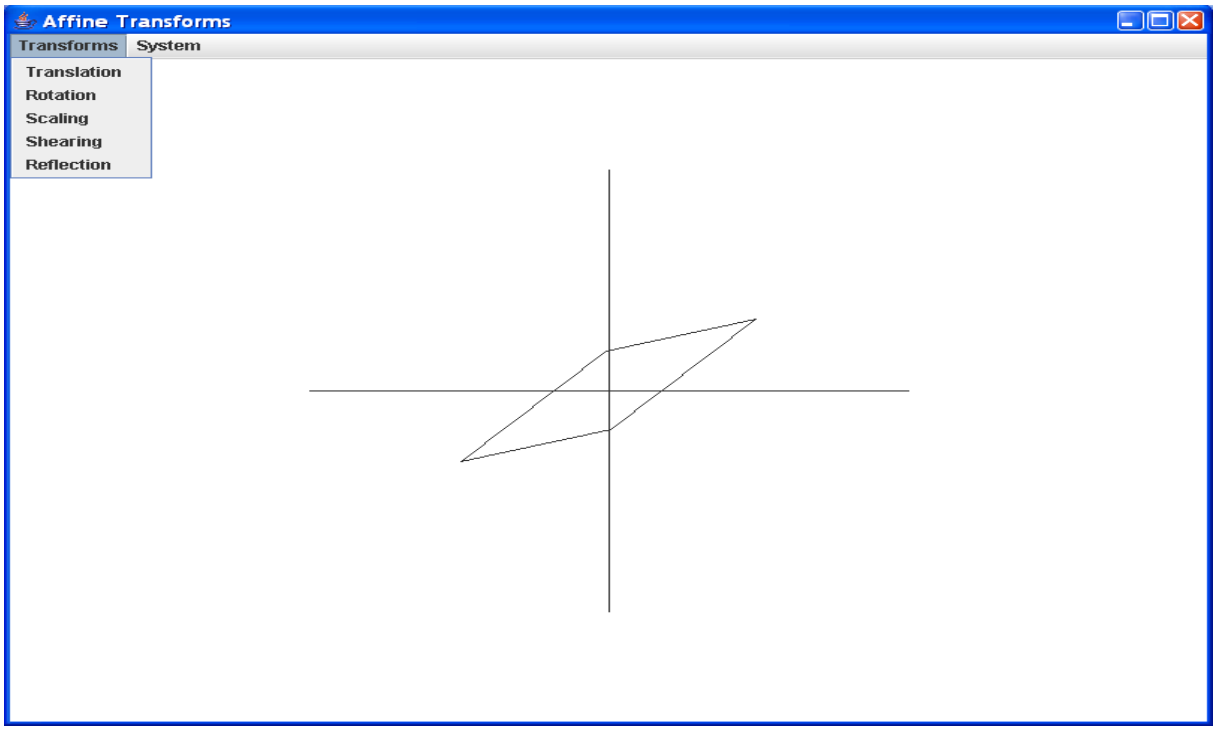
```
            panel.transformType = panel.TRANSLATION;
        } else if ("Rotation".equals(command)) {
            panel.transformType = panel.ROTATION;
        } else if ("Scaling".equals(command)) {
            panel.transformType = panel.SCALING;
        }
    }
}

class    TransformPanel    extends    JPanel    implements    MouseListener,
MouseMotionListener {

    static final int NONE = 0;
    static final int TRANSLATION = 1;
    static final int ROTATION = 2;
    static final int SCALING = 3;
    int transformType = NONE;
    Shape drawShape = null;
    Shape tempShape = null;
    Point p = null;
    int x0 = 400;
    int y0 = 300;

    public TransformPanel() {
        super();
        setPreferredSize(new Dimension(800, 600));
        setBackground(Color.white);
        drawShape = new Rectangle(-50, -50, 100, 100);
        addMouseListener(this);
        addMouseMotionListener(this);
    }

    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.translate(x0, y0);
        g2.drawLine(-200, 0, 200, 0);
        g2.drawLine(0, -200, 0, 200);
        g2.draw(drawShape);
    }

    public void mouseClicked(MouseEvent ev) {
    }

    public void mouseEntered(MouseEvent ev) {
    }

    public void mouseExited(MouseEvent ev) {
    }

    public void mousePressed(MouseEvent ev) {
        p = ev.getPoint();
    }

    public void mouseReleased(MouseEvent ev) {
        Graphics g = getGraphics();
        Point p1 = ev.getPoint();
        AffineTransform tr = new AffineTransform();
        switch (transformType) {
            case TRANSLATION:
                tr.setToTranslation(p1.x - p.x, p1.y - p.y);
```

```
                break;
            case ROTATION:
                double a = Math.atan2(p1.y - y0, p1.x - x0) - Math.atan2(p.y
- y0, p.x - x0);
                tr.setToRotation(a);
                break;
            case SCALING:
                double sx = Math.abs((double) (p1.x - x0) / (p.x - x0));
                double sy = Math.abs((double) (p1.y - y0) / (p.y - y0));
                tr.setToScale(sx, sy);
                break;
        }
        drawShape = tr.createTransformedShape(drawShape);
        repaint();
    }

    public void mouseMoved(MouseEvent ev) {
    }

    public void mouseDragged(MouseEvent ev) {
        Point p1 = ev.getPoint();
        AffineTransform tr = new AffineTransform();
        switch (transformType) {
            case TRANSLATION:
                tr.setToTranslation(p1.x - p.x, p1.y - p.y);
                break;
            case ROTATION:
                double a = Math.atan2(p1.y - y0, p1.x - x0) - Math.atan2(p.y
- y0, p.x - x0);
                tr.setToRotation(a);
                break;
            case SCALING:
                double sx = Math.abs((double) (p1.x - x0) / (p.x - x0));
                double sy = Math.abs((double) (p1.y - y0) / (p.y - y0));
                tr.setToScale(sx, sy);
                break;
         }
        Graphics2D g = (Graphics2D) getGraphics();
        g.setXORMode(Color.white);
        g.translate(x0, y0);
        if (tempShape != null) {
            g.draw(tempShape);
        }
        tempShape = tr.createTransformedShape(drawShape);
        g.draw(tempShape);
    }
}
```

Exercise 2: Copy the source above source code and modify:

- Add one more menu “System”, when click here there is an Item “Close” to exit program.

- Add more 2 objects (Shearing and Reflection) onto the menu "Transforms" and therefore user can use them later. The result is as below figure:



Hint: The case of Shearing:

```
double shx = ((double) (p1.x - x0) / (p.x - x0)) - 1;
double shy = ((double) (p1.y - y0) / (p.y - y0)) - 1;
tr.setToShear(shx, shy);
```

The case of Reflection: tr.setTransform(-1, 0, 0, 1, 0, 0); (you can try with the case of y reflection)

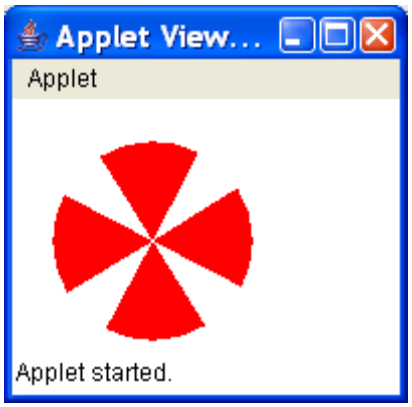
Part 3: Animation

Base on the below source code to draw a wheel:

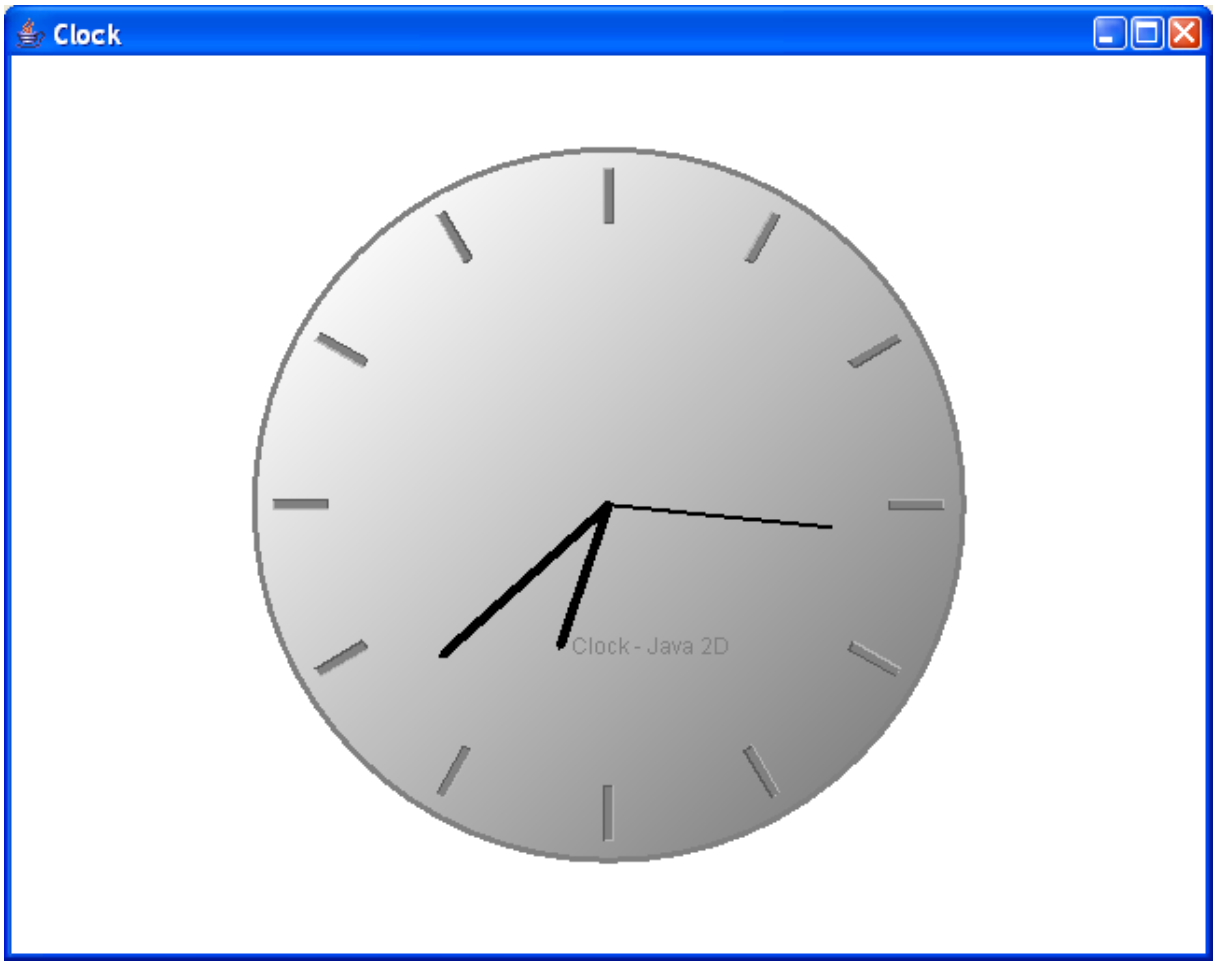
```
import java.applet.Applet;
import java.awt.*;
public class Wheels extends Applet implements Runnable {
    Thread Xeq;
    int StartAngle;
    int Delay;
    public void init() {
        StartAngle = 0;
        Delay = 40;
    }
```

```
    public void start() {
        if (Xeq == null) {
            Xeq = new Thread(this);
            Xeq.start();
        }
    }
    public void run() {
        while (true) {
            StartAngle = StartAngle - 1;
            repaint();
            try {
                Xeq.sleep(Delay);
            } catch (InterruptedException ex) {
            }
        }
    }
    public void paint(Graphics g) {
        int Blade = 0;
        g.setColor(Color.red);
        while (Blade < 4) {
            g.fillArc(20, 20, 100, 100,
                    StartAngle + Blade * 90, 60);
            ++Blade;
        }
    }
}
```

The result:



Exercise 3: **Draw a Clock as below but change all "|" = number (from 1 to 12)**



Hint: this is the source code

```
public class ClockTest extends JApplet {
  public static void main(String args[]) {
    JFrame frame = new JFrame();
    frame.setTitle("Clock");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    JApplet applet = new ClockTest();
    applet.init();
    frame.getContentPane().add(applet);
    frame.pack();
    frame.setVisible(true);
  }
  public void init() {
    JPanel panel = new ClockPanel();
    getContentPane().add(panel);
  }
}
```

```
class ClockPanel extends JPanel implements ActionListener{
  AffineTransform rotH = new AffineTransform();
  AffineTransform rotM = new AffineTransform();
  AffineTransform rotS = new AffineTransform();
  public ClockPanel() {
    setPreferredSize(new Dimension(640, 480));
    setBackground(Color.white);
    Timer timer = new Timer(500, this);
    timer.start();
  }
  public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D)g;
    g2.translate(320,240);
    // clock face
    Paint paint = new GradientPaint(-150,-
150,Color.white,150,150,Color.gray);
    g2.setPaint(paint);
    g2.fillOval(-190, -190, 380, 380);
    g2.setColor(Color.gray);
    g2.drawString("Clock - Java 2D", -20, 80);
    Stroke stroke = new BasicStroke(3);
    g2.setStroke(stroke);
    g2.drawOval(-190, -190, 380, 380);
    for (int i = 0; i < 12; i++) {
      g2.rotate(2*Math.PI/12);
      g2.fill3DRect(-3, -180, 6, 30, true);
    }
    // clock hands
    Shape hour = new Line2D.Double(0, 0, 0, -80);
    hour = rotH.createTransformedShape(hour);
    Shape minute = new Line2D.Double(0, 0, 0, -120);
    minute = rotM.createTransformedShape(minute);
    Shape second = new Line2D.Double(0, 0, 0, -120);
    second = rotS.createTransformedShape(second);
    g2.setColor(Color.black);
    g2.setStroke(new BasicStroke(5, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
    g2.draw(hour);
    g2.draw(minute);
    g2.setStroke(new BasicStroke(2));
```

```
    g2.draw(second);
  }
  public void actionPerformed(ActionEvent e) {
    int hour = Calendar.getInstance().get(Calendar.HOUR);
    int min = Calendar.getInstance().get(Calendar.MINUTE);
    int sec = Calendar.getInstance().get(Calendar.SECOND);
    rotH.setToRotation(Math.PI * (hour+min/60.0)/6.0);
    rotM.setToRotation(Math.PI * min /30.0);
    rotS.setToRotation(Math.PI * sec /30.0);
    repaint();
  }
}
```

Homework:

1. Combine ex1 and ex2 in order to have a full program with 3 menus on the menu bar: (Choose Objects, Transforms, System)

2. Draw a car (Draw a wheel of a car, Rotate by timer, Move along x-axis) **by using OpenGL on Java (Eclipse or Netbeans IDE, version <=7.0)**